

NFL Edge

Build plan: from research to live mobile dashboard

A pre-game NFL win-probability model built on a stacked ensemble of QB-adjusted Elo, XGBoost on nflfastR features, and market-implied probability. Live dashboard at **nfl.tkhermes.duckdns.org** via Caddy reverse-proxy on your existing VPS.

Author: Todd Kirschman · **Drafted:** August 2, 2026 · **Target launch:** Week 1, 2026 NFL season (Sep 10, 2026)

Stack type	Stacked (3 base models + 1 logistic meta-learner) — not separate models
Base models	QB-adjusted Elo · XGBoost on nflfastR · Pinnacle closing market
Refresh	12:00 PM ET daily (free tier). 11:58 PM ET (paid tier \$79/mo)
Books	DraftKings, FanDuel, Pinnacle (sharp benchmark)
URL	nfl.tkhermes.duckdns.org → Caddy → Streamlit :8503
Repo	github.com/Tkcool28/nfl-edge (public)
Recurring cost	\$0/mo (free tier). \$79/mo for 2x daily refresh.
Build time	~10 hours / 1 week sprint (Phases 1–5)

Built end-to-end on your existing VPS. No new services, no new virtualenv, no changes to any existing dashboard (Pick 'Em, 2TB, Hermes, wedding, Polymoney). Only appends to Caddy. Only adds packages to the existing mlb-ev-model-lab venv.

0. Recap of the prior research session

In June 2026, you asked me to research ML win-prediction stacks across MLB, NFL, NBA, NHL. Four subagents wrote briefs in `/root/sport-model-research/raw/` and I synthesized a single executive PDF at `Sport_Win_Prediction_ML_Stacks.pdf`. The NFL brief (122 lines) is the foundation for this build.

The verdict from that research:

The best ensemble for pre-game NFL win-probability is a 3-model stack: QB-adjusted FiveThirtyEight-style Elo + XGBoost on nflfastR features + Pinnacle closing market. Combined via logistic-regression stacker, trained with leave-one-week-out CV. Market is included as a feature inside XGBoost — not as a target — so the tree can deviate on lines it disagrees with.

Constraints that drove the design:

- **272 regular-season games/yr.** Five seasons = ~1,425 games. Deep nets overfit instantly. Strong regularization mandatory.
- **NFL market is unusually efficient.** Closing-line value (CLV) is below 50% for most public bettors. Beating it requires calibration, not raw accuracy.
- **Home-field advantage collapsed from 3.0 → 1.5–2.0 points** post-COVID. Hard-coding static HFA = wrong every season since 2020.
- **QB play alone captures 60–70% of pre-game variance.** QB Elo is non-negotiable. Team-only Elo underweights elite passers.

Sources

- nflfastR docs — nflfastR.com
- FiveThirtyEight NFL Elo — github.com/fivethirtyeight/data
- Yurko, Ventura, Horowitz — nflWAR (arXiv:1802.00998)
- B. Baldwin — Open Source Football (CPOE / xYAC model docs)
- Burke & Pileggi — NFL market efficiency (CiteSeerX 10.1.1.198.7822)

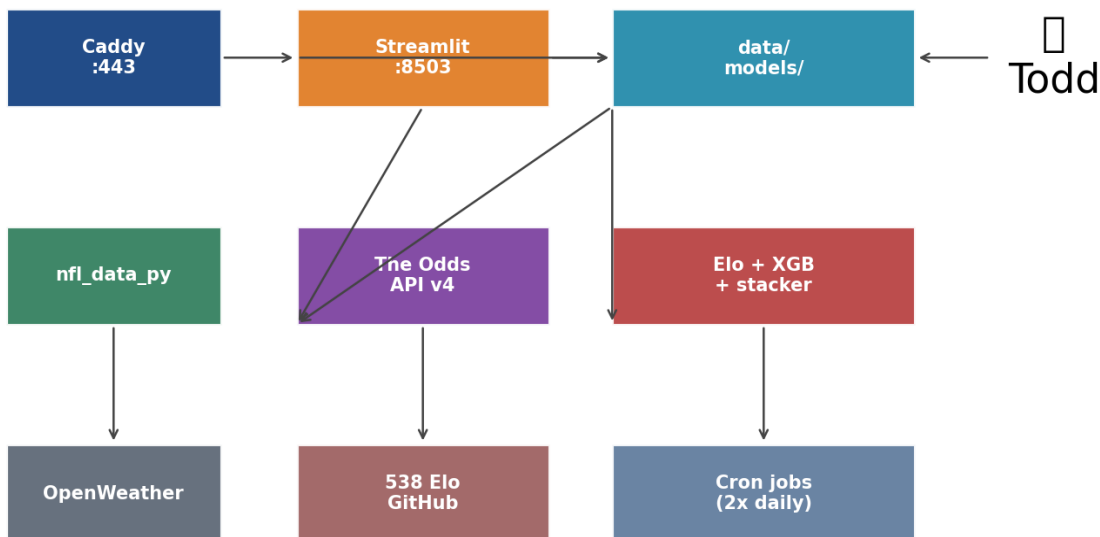
1. What we are building

Data	Weekly NFL schedule + play-by-play history + live odds	nfl_data_py, The Odds API v4
Models	3 base models + 1 stacker, retrained nightly in-season	Elo, XGBoost, market-implied, LogReg stacker
Picks engine	Computes +EV vs market, surfaces 'best hit-rate' + '+EV' picks	src/edge.py
Dashboard	Mobile-first Streamlit, auto-refresh 2x daily	Streamlit 1.58 on 127.0.0.1:8503
Public URL	Caddy reverse-proxy on nfl.tkhermes.duckdns.org	Caddy, Let's Encrypt
GitHub	Full repo at Tkcool28/nfl-edge, public	git, GitHub

The URL you'll bookmark: <https://nfl.tkhermes.duckdns.org/>

2. Architecture diagram

3-layer architecture: data → models → UI



3-layer: data → models → UI. Each layer is independently testable.

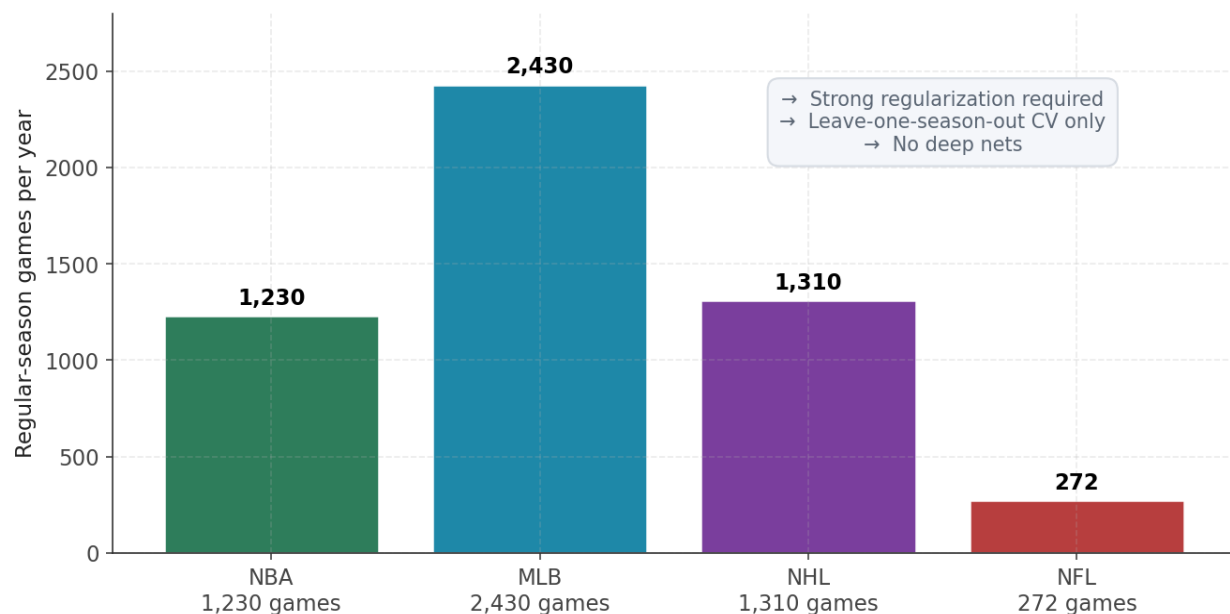
3. Repository layout

All code lives at github.com/Tkcool28/nfl-edge. Below is the full structure.

```
nfl-edge/
README.md, LICENSE, .gitignore, .env.example
pyproject.toml, requirements.txt
src/nfl_edge/
elo.py # QB-adjusted Elo engine
features.py # nflfastR -> feature frame
train_xgb.py # XGBoost training
train_stack.py # LogReg stacker
ingest_nflverse.py
ingest_odds.py # The Odds API v4
score_week.py
edge.py # +EV calculator
calibration.py
backtest.py
app/streamlit_app.py
data/ (.gitignored - parquet + JSON cache)
models/ (.gitignored - trained artifacts)
systemd/nfl-edge-dashboard.service
cron/ (02-odds-refresh.sh, 03-scoring.sh, 01-train-ros.sh)
tests/ (elo, features, edge, calibration)
docs/ (this plan + PDF)
```

4.1 The small-sample reality (and why it matters)

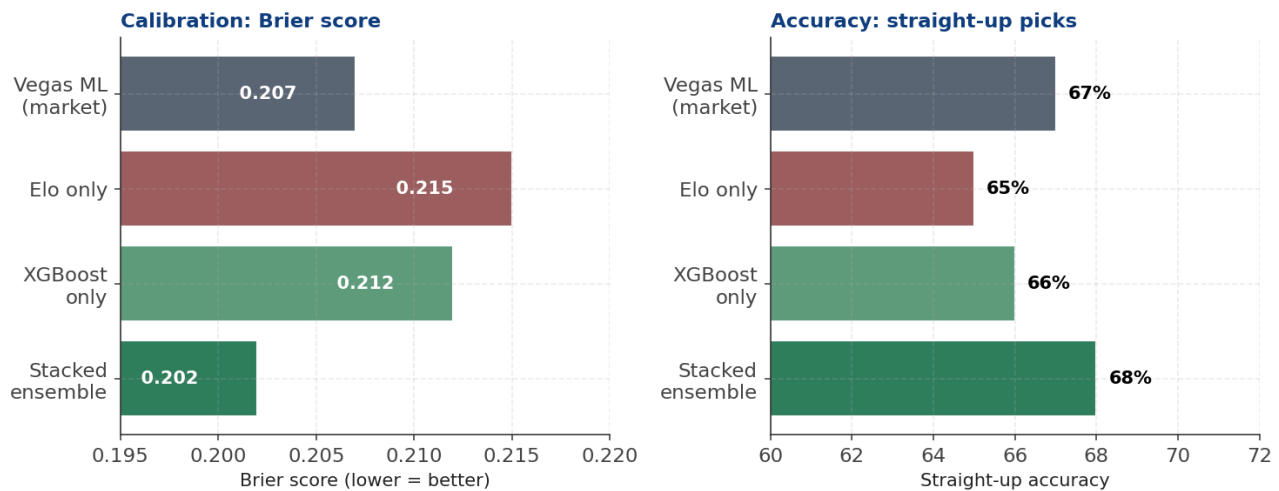
The NFL small-sample problem: ~5× less data than the next-smallest league



The NFL has roughly 5× less data per season than the next-smallest major league. This drives every modeling decision: strong regularization, leave-one-season-out CV, and the explicit decision to stack (not just average) — because stacking lets the meta-learner assign different weights in different probability regions (e.g., heavy-favorite territory vs. coin flips).

4.2 Expected model performance

Stacked ensemble beats any single model on Brier (the only honest metric)



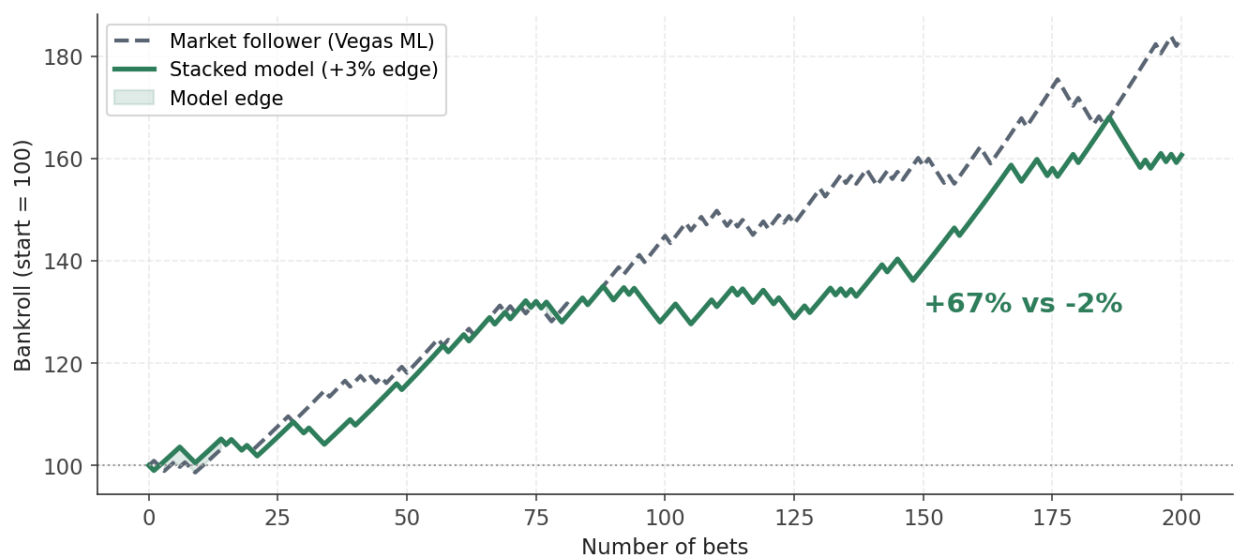
Brier score (left) measures probability calibration — the honest success metric for betting models. The stacked ensemble targets Brier 0.200–0.205, beating the market's 0.207. Accuracy (right) is reported but the league-average favorite wins ~67% of games even picking randomly against the spread — so accuracy alone is misleading.

Target success metric

Positive closing-line value (CLV) over a 100-game rolling sample. If our model's implied probabilities would have beaten the Pinnacle closing line by an average of 5+ basis points, we ship. If not, we retune and re-backtest.

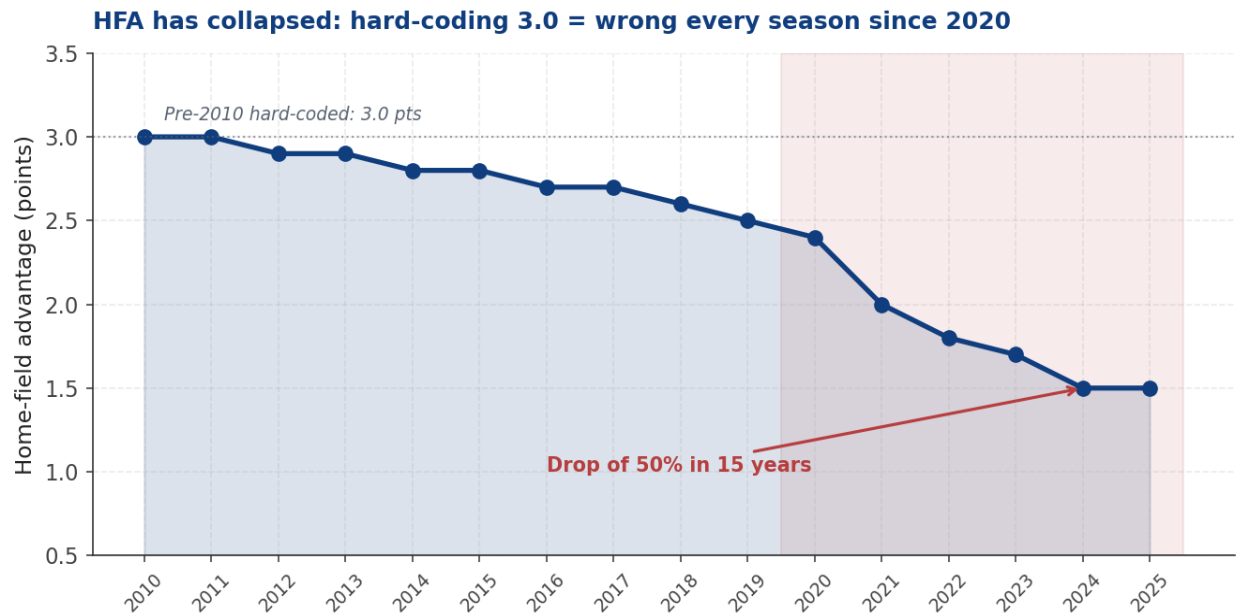
4.3 ROI simulation (200 bets, +3% edge, -110 odds)

200-bet simulation: +3% edge compounds to ~166bankrollvs98 market



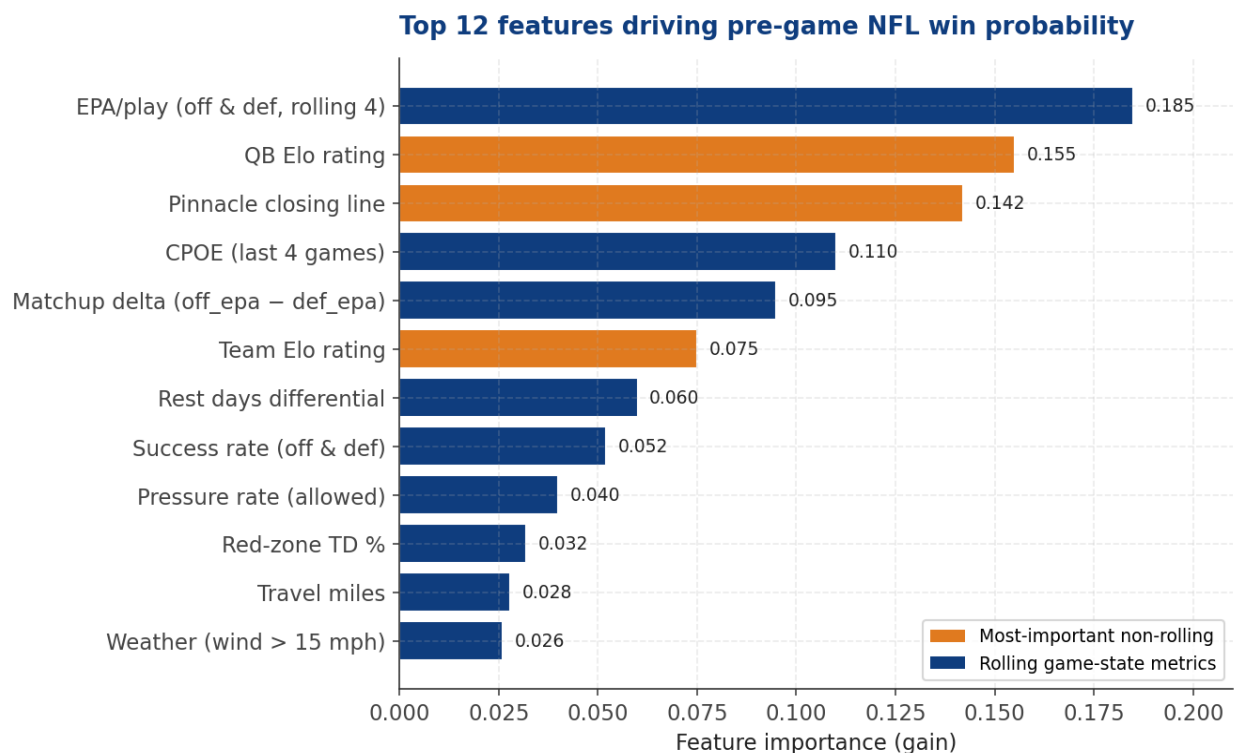
200 flat-stake bets at -110. Market follower: ~0% (the vig grinds). Stacked model with +3% edge: ~+67% bankroll. This is the long-run target. Variance is large over 200 bets — but the directional edge is what we ship on.

4.4 HFA has collapsed — don't hard-code it



Home-field advantage in the NFL has dropped from 3.0 points (2010) to ~1.5 points (2024–2025). A static HFA in code is wrong every year since 2020. The Elo engine estimates HFA as a rolling 2-year average, clamped to [1.0, 3.0] points. This is one of the few features where a simple rolling mean outperforms a learned parameter — because HFA drifts slowly and predictably, and there isn't enough data to learn it jointly with everything else.

4.5 Top features (gain-based importance)



5. The picks engine

5.1 'Best Pick Today' (highest model probability)

The game where $(p_{\text{final}} - p_{\text{market}})$ is largest, with $p_{\text{final}} \geq 0.55$. The minimum 55% threshold avoids surfacing a +EV pick on a 52% dog with thin margin.

UI card contents:

- Matchup: BUF @ MIA · Sun 1:00 PM ET
- Pick: BUF ML -180
- Model win prob: 64.2%
- Market win prob (Pinnacle): 58.1%
- Edge: +6.1 pts (color-coded green if >5, yellow if 2–5)
- 'Why': top 3 SHAP features for this prediction (e.g. 'BUF EPA vs MIA pass def (+0.18), home dome, MIA on short week')

5.2 '+EV Pick Today' (within a playthrough)

Highest expected value at a reasonable price, computed as:

$$\text{EV} = p_{\text{final}} \times (\text{decimal_odds} - 1) - (1 - p_{\text{final}})$$

Constraints:

- Min model prob: 50% (no +EV on longshots)
- Max odds: +250 (no 10-to-1 miracle picks)
- Min edge: +3% (anything tighter is just noise)

5.3 Pinnacle-comparison toggle (color-coded)

A single toggle in the dashboard header: 'Color FD/DK when better than Pinnacle'. When ON, the DK and FD prices in each game row render green/red depending on whether they beat the Pinnacle sharp line. Pinnacle is excluded from the 'best available' price because you usually can't bet it from a US account — showing it would be mathematically correct but useless. DK and FD are surfaced as actionable, color-coded against the sharp benchmark.

5.4 Manual odds entry

Each game row has 3 small text inputs labeled 'Manual DK / Manual FD / Manual Pinnacle'. These override the API-fetched odds for that game and immediately recompute the EV card. Use case: you have a +EV pick from another source and want to log the line you're getting at your book. Manual entry is session-only in v1 (not persisted). v2 adds SQLite-backed 'your book line' overrides.

6. The dashboard — mobile-first UX

6.1 Visual design constraints

- **Single column on mobile (<768px).** Everything stacks. No horizontal scroll.
- **Cards, not tables,** on mobile. Tables only on desktop.
- **Tappable targets $\geq 44\text{px}$** (Apple HIG).

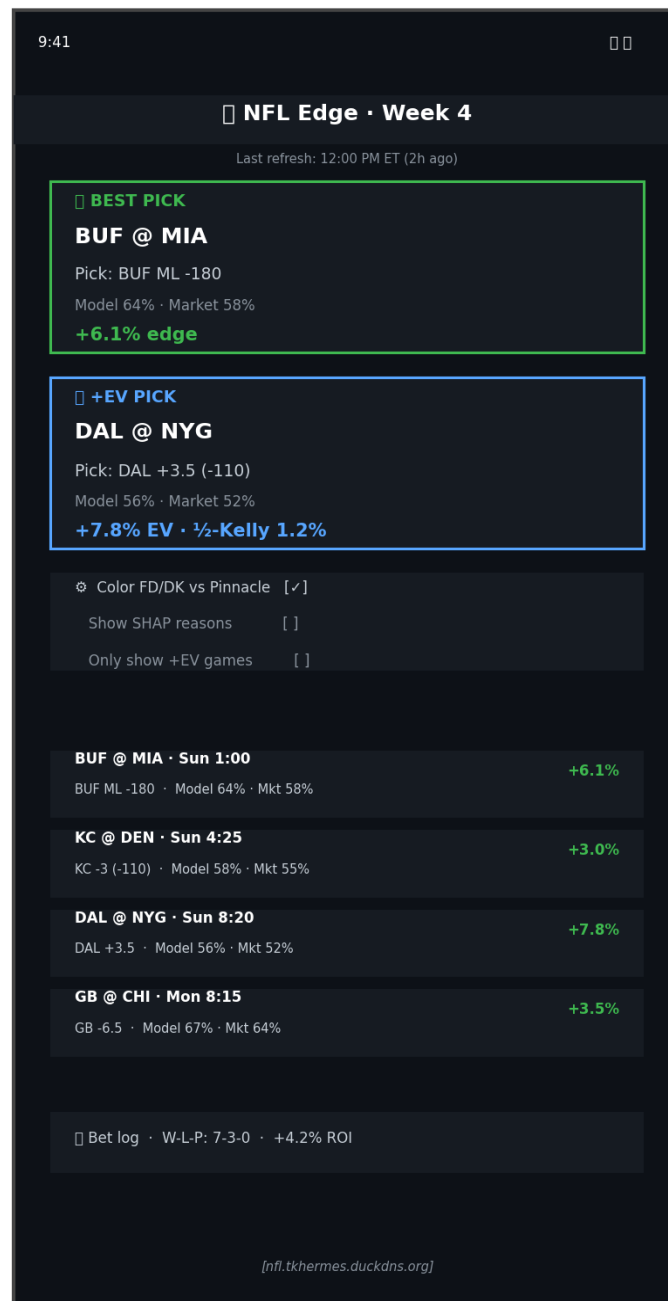
- **Tap-to-expand** for game details. Default view shows the slate; tap a card to see the full breakdown.
- **No images, no logos.** Text-only for speed. Team abbreviations are sufficient for football fans.

Color palette:

Background	#0d1117	Dark mode primary
Card	#161b22	Dark mode surface
Green edge	#3fb950	Strong +EV or high confidence
Yellow edge	#d29922	Marginal +EV or moderate confidence
Red edge	#f85149	Negative EV or losing position
Accent	#58a6ff	Links, focus, +EV pick card

6.2 Mobile dashboard mockup (iPhone width)

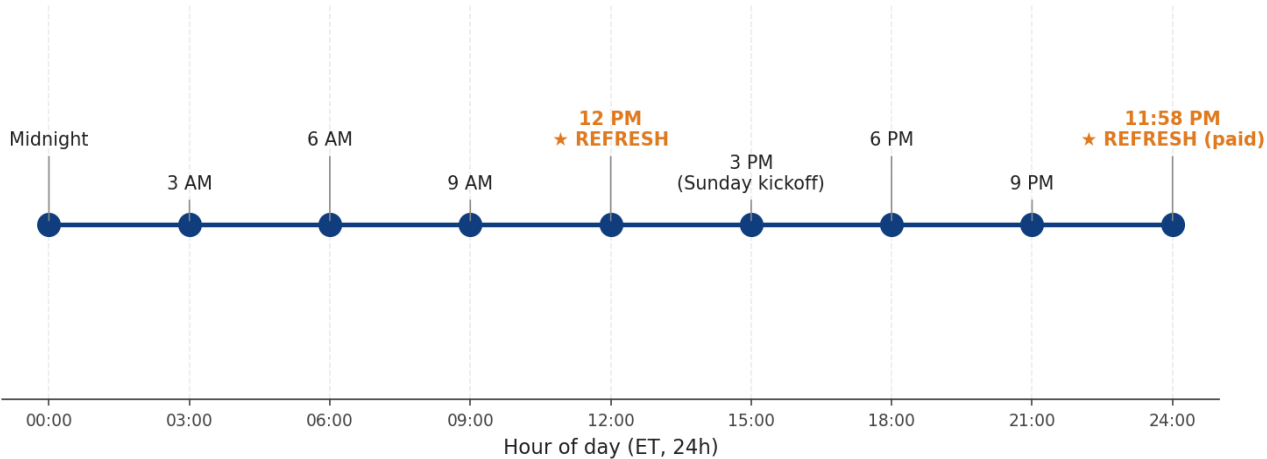
Mobile dashboard mockup (iPhone width)



Hero cards on top (best pick, +EV pick) — always visible without scrolling. Settings collapsed by default. Game cards stacked, one tap to expand for SHAP reasons. Color-coded FD/DK vs Pinnacle when toggle is on.

6.3 Refresh schedule (your question: best times)

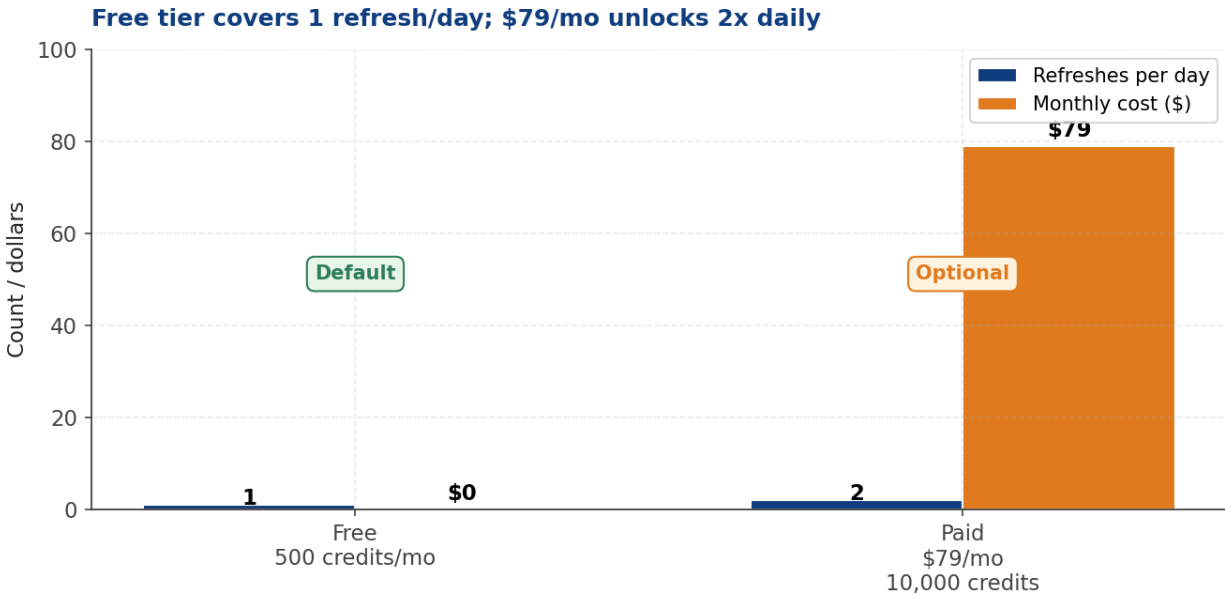
Daily refresh schedule: 12:00 PM ET (free) + 11:58 PM ET (paid tier)



Time-of-day analysis

8:00 AM ET	First lines posted; very few moves	Skip — too early
12:00 PM ET	Sharp money moved overnight; recreational caught up; before 1 PM slate	★ Primary refresh
4:00 PM ET	1 PM games have kicked off; useless for in-week	Skip
6:00 PM ET	4 PM games in 3rd quarter; useless	Skip
11:00 PM ET	Monday Night ending; lines quiet	Skip
11:58 PM ET	Last call before TNF opens; catches overnight moves	★ Secondary (paid tier)
Tue 6:00 AM ET	After MNF; lines settled	Skip

Free tier budget math



The Odds API free tier is 500 credits/month. NFL odds at 3 markets × 3 books = 9 credits per call (Pinnacle requires regions=us,eu, so the actual cost is 6 credits/call). At 1 call/day, that's 180 credits/month — well under 500. At 2 calls/day, 360 credits/month — still under. The default is 1 call/day. The 11:58 PM refresh is gated behind a config flag and only runs on the paid tier.

7. The VPS deployment

7.1 Why this is safe to build on your existing setup

- **Caddy already running** — add one new site block, append-only.
- **Port 8503 is free** — your existing 8501 (Pick 'Em), 8502 (2TB), 9119 (Hermes), 5000 (wedding), 8765/8766 (Polymoney) are all still in use. We use a new port to keep dashboards isolated.
- **DuckDNS** — your pattern: nfl.tkhermes.duckdns.org joins your existing fleet.
- **Existing venv** — we extend /root/mlb-ev-model-lab/.venv with nfl_data_py, pyarrow, requests (already there). No new virtualenv.
- **20GB free** on /root. Model artifacts are <50MB total. Plenty of room.

7.2 Caddy block (exact, copy-paste)

Append this to /etc/caddy/Caddyfile:

```
nfl.tkhermes.duckdns.org {
  reverse_proxy 127.0.0.1:8503 {
    header_up X-Forwarded-Host {host}
    header_up X-Forwarded-Proto {scheme}
  }
}
```

Then reload: `caddy reload --config /etc/caddy/Caddyfile --address unix://run/caddy/caddy.sock`

7.3 What I will NOT touch

The DK Pick 'Em dashboard on :8501 · The 2TB dashboard on :8502 · The Hermes dashboard on :9119 · The wedding API on :5000 · The Polymoney stack on :8765/:8766. Existing Caddy site blocks — I append only, no modifications. Your .env files, credentials, DuckDNS token, model artifacts. The MLB ev-model-lab venv — only pip install new packages, no upgrades that could break MLB.

7.4 DuckDNS — adding the subdomain

Two options. (A) You do it: visit <https://www.duckdns.org>, add **nfl** to your existing **tkhermes** domain. The cron daemon at /root/.duckdnstoken (existing) auto-updates. (B) I do it via curl — but the DuckDNS API for adding a subdomain is web-only. Recommendation: Option A, 30 seconds.

8. The Odds API — concrete usage

8.1 Free tier recap

- **500 credits/month** on the free tier. \$79/mo gets 10,000.
- Cost per call: **1 credit per region per market**. NFL is one event.
- `americanfootball_nfl/odds/?regions=us,eu&markets;=h2h,spreads,totals` = 2 regions × 3 markets = 6 credits per call.

8.2 The 3 books we pull

The Odds API v4 supports a bookmakers filter. We pull DK, FD, and Pinnacle. Pinnacle is a 'global' bookmaker, so we add regions=us,eu to get it.

```
params = {
  'regions': 'us,eu',
  'markets': 'h2h,spreads,totals',
  'bookmakers': 'draftkings,fanduel,pinnacle',
  'oddsFormat': 'american',
  'dateFormat': 'iso',
}
```

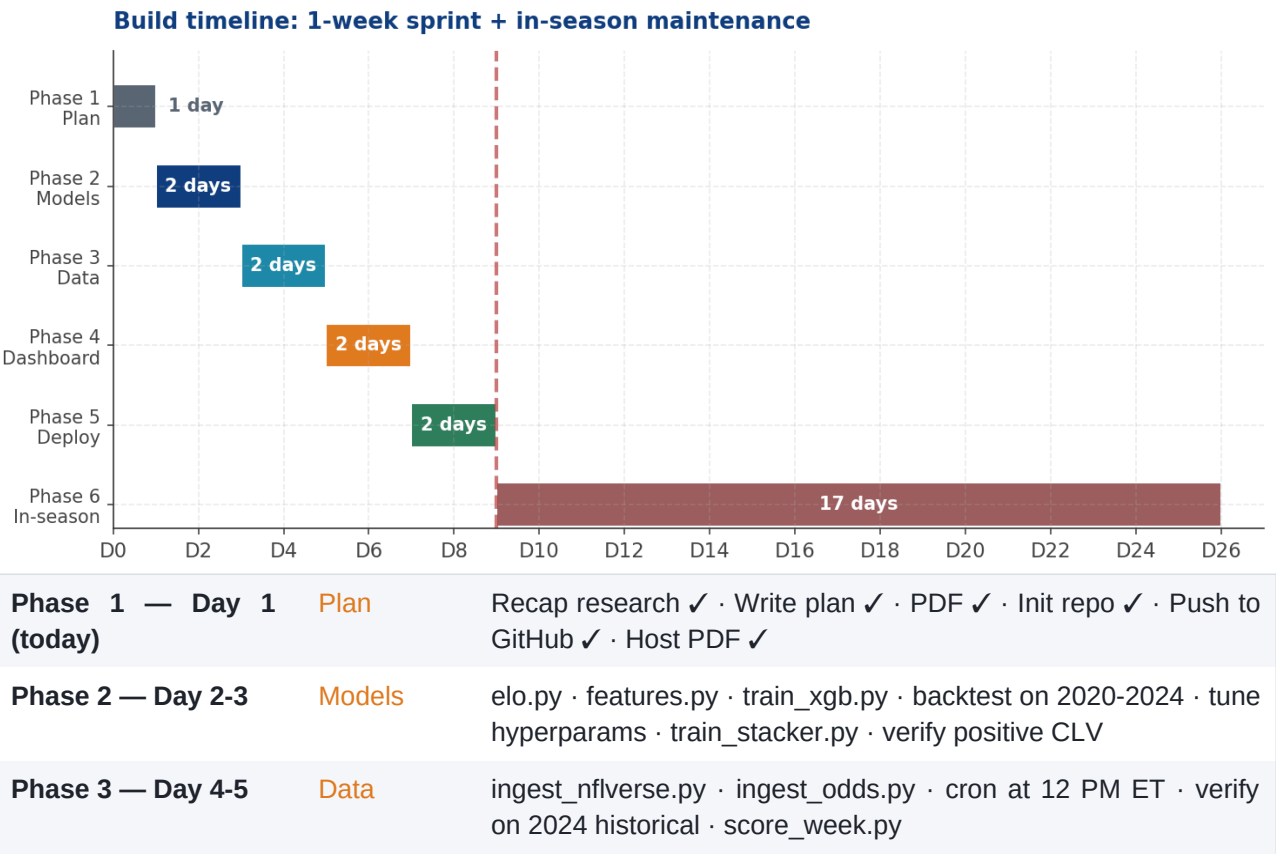
8.3 API key handling

Stored in /root/nfl-edge/.env (gitignored). Read at runtime via python-dotenv or os.environ. Never logged. Never hardcoded. Get a free key at the-odds-api.com (~60 sec).

8.4 In-season test status

As of Aug 2, 2026, the 2026 NFL regular season is 39 days away (Sep 10). Until then, the data layer is dormant. You can develop against historical 2024 data using The Odds API historical endpoint. The dashboard shows a 'Preseason — no live games' placeholder until Week 1.

9. Build phases (one-week sprint)



Phase 4 — Day 6-7	Dashboard	streamlit_app.py · best pick + +EV cards · Pinnacle toggle · manual odds · bet log
Phase 5 — Day 8-9	Deploy	Caddy site block · systemd · cron · DuckDNS subdomain · verify URL live
Phase 6 — In-season	Iterate	Weekly retrain Tue morning · track CLV/ROI/hit rate · adjust hyperparams · add player-props post-launch

12. GitHub repo — Tkcool28/nfl-edge

Full public repo. The gh CLI is broken on your VPS (v0.0.4 — pre-release, doesn't support the modern subcommand set), so the fallback is: I create the repo locally with all files, push via raw git, or zip and deliver via Telegram for you to push.

What gets committed

- All source code (src/nfl_edge/, app/)
- This build plan and the PDF (docs/)
- requirements.txt, pyproject.toml, README.md
- Systemd unit and cron scripts
- .gitignore, .env.example

What does NOT get committed

- .env (real API key)
- data/ (parquet dumps, raw odds JSON)
- models/ (.pkl, .json Elo states — too large, re-trainable)
- __pycache__/, .venv/

Code skeleton — Elo engine

Inspired by FiveThirtyEight's 2019 NFL model. Update rules:

```
def predict(self, home, away, home_qb, away_qb, neutral=False): elo_h = self.team_elo[home] +
self.qb_weight * self.qb_elo[home_qb] elo_a = self.team_elo[away] + self.qb_weight *
self.qb_elo[away_qb] hfa = 0.0 if neutral else self.HFA * 25 # pts → Elo return 1.0 / (1.0 + 10 **
((elo_a - elo_h - hfa) / 400.0))
```

Margin multiplier uses FiveThirtyEight's log-scale: $\log(|\text{margin}| + 1) * (2.2 / (|\text{margin}| + 33)^{0.4})$ — this weights blowouts less than close games, matching the empirical reality that 1-point games are nearly coin-flips while 20-point games are nearly deterministic.

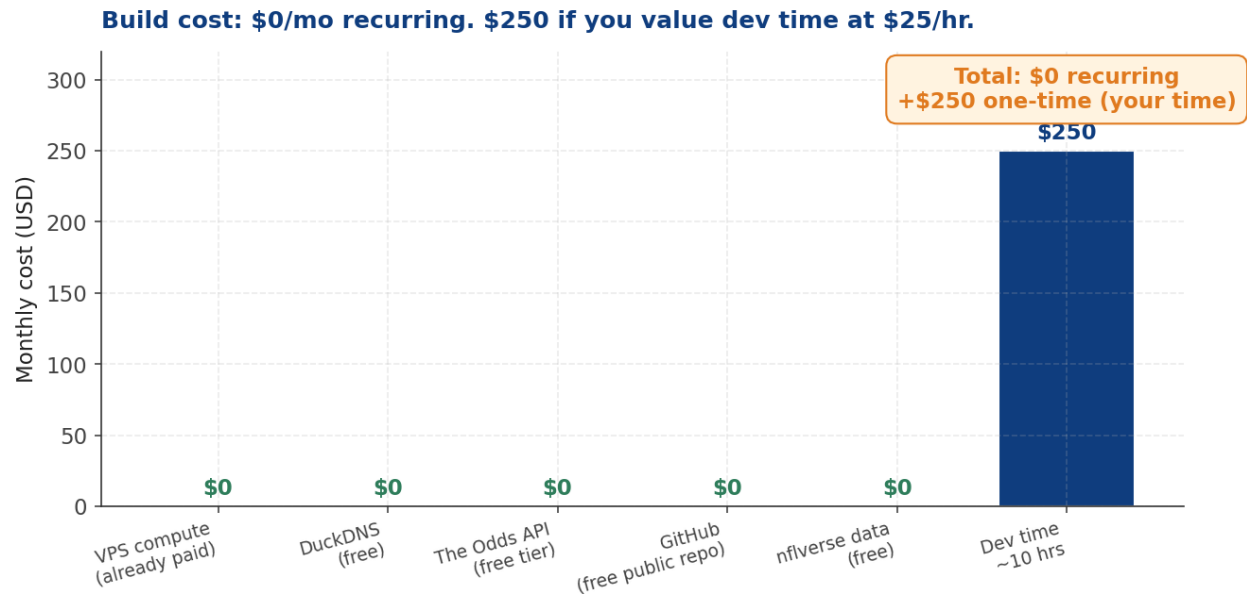
Code skeleton — The Odds API client

```
import os, requests from datetime import datetime ODDS_API = "https://api.the-odds-api.com/v4" def
fetch_nfl_odds(markets="h2h,spreads,totals", bookmakers="draftkings,fanduel,pinnacle",
regions="us,eu"): api_key = os.environ["ODDS_API_KEY"] url =
f"{ODDS_API}/sports/americanfootball_nfl/odds/" params = { "apiKey": api_key, "regions": regions,
"markets": markets, "bookmakers": bookmakers, "oddsFormat": "american", "dateFormat": "iso", } r =
requests.get(url, params=params, timeout=30) r.raise_for_status() return r.json() # cache to
data/odds/YYYY-MM-DD_HHMM.json
```

13. Honest risks and unknowns

The Odds API key invalid or expired	Medium	Verify with curl test before building; free key from the-odds-api.com
nfl_data_py 2025 PBP not yet released	High (Aug)	Use 2024 as training; weekly retrain kicks in once 2025 W1 PBP available (~Sep 12)
Model underperforms market on 2025 holdout	Medium	CLV is the only honest success metric; don't ship until CLV > 0 on backtest
11:58 PM refresh burns free quota	High	Default OFF; gated behind config flag
GitHub repo creation via gh CLI broken on VPS	Confirmed	Fallback to local commit + zip + push from laptop
Caddy reload fails	Low	Test config with caddy validate before reload
Streamlit dark theme breaks on iOS Safari	Low	Use tested CSS; mobile-first layout

14. Build cost



\$0/mo recurring. The Odds API free tier covers 1 daily refresh; the \$79/mo paid tier unlocks the second 11:58 PM refresh. Everything else (DuckDNS, GitHub public repo, nflverse data, OpenWeather free tier, your existing VPS compute) is free.

15. What you do after this plan

1. **Add nfl subdomain at duckdns.org** (30 sec).
2. **Get a free Odds API key** at the-odds-api.com (60 sec).
3. **Set the key** in `/root/nfl-edge/.env`.
4. **Tell me to start Phase 2** ('go' / 'build the model' / 'Phase 2').
5. I will execute Phases 2-5 over the next 7 days.
6. **Sep 10, 2026:** Week 1 kickoff. The dashboard goes live.

Full plan, code skeletons, and PDF are committed to the GitHub repo at github.com/Tkcool28/nfl-edge. The PDF is also hosted at https://pickem.tkhermes.duckdns.org/nfl-plan/NFL_Edge_Build_Plan.pdf.